

DATA-236 - Distributed Systems for Data Engineering

Homework 5

Total Points: 25

Overview

Create a RESTful API for a Library Management System using FastAPI and MySQL with full CRUD operations for books and authors.

Requirements:

Database Design (3 points)

Design two related tables.

The Authors table should include id (primary key), first_name, last_name, email (unique), and timestamps.

The Books table should contain id (primary key), title, isbn (unique), publication_year, available_copies (default 1), author_id (foreign key), and timestamps. Use appropriate data types and constraints.

API Implementation (5 points)

Implement CRUD endpoints for both entities.

For authors, create endpoints to add new authors, retrieve all authors with pagination, get individual authors, update author information, and delete authors.

For books, provide similar functionality including creating books, listing all books, retrieving specific books, updating book details, and removing books.

Include one additional endpoint to get all books written by a specific author.

Data Validation and Error Handling (2 points)

Use Pydantic schemas for request and response validation.

Implement proper HTTP status codes and error messages for scenarios like resource not found, validation errors, and constraint violations.

Ensure email format validation and prevent deletion of authors who have associated books.

Submit screenshots of each action tested using API endpoints. Submit a screenshot of your DB

Frontend Integration with Redux (5 points)

In this part, you will build a React + Redux frontend that integrates with the FastAPI + MySQL backend you created in Part I. The frontend should use Axios for API calls and Redux Toolkit to manage application state.

I. Redux Setup (1 point)

- Create a Redux store using Redux Toolkit.
- Define a booksSlice to manage the state of books.
- Include reducers/actions for fetching, adding, updating, and deleting books.

II. Async Thunks with Axios (1 point)

- Implement async thunks that call your FastAPI endpoints from Part I:
 - fetchBooks → GET /books
 - createBook → POST /books
 - updateBook → PUT /books/{id}
 - deleteBook → DELETE /books/{id}
- Handle success and error cases properly.

III. Home Screen (1 point)

- Display all books by reading from Redux state.
- The list should update automatically when books are created, updated, or deleted.

IV. Create / Update Screens (1 point)

- Build a form to create a new book.
- Build a form to update an existing book (select a book by ID).
- On submission, dispatch the correct Redux thunk and show the updated results on the Home Screen.

V. Delete Functionality (1 point)

- Add a delete button next to each book in the list.
- On click, dispatch the delete thunk and update the Redux state to remove the book.

Submission Guidelines for Redux Part

- Submit one screenshot per feature (Home, Create, Update, Delete) showing the code snippet (Redux thunk or slice) and the corresponding UI output together.
- Make sure the screenshot includes both code and browser output in the same frame (side-by-side or one above the other).
- Do not submit full files. Only the relevant code + working result.

PART 2

Homework: Build a MCP server that fetches recipes from TheMealDB (10 points)

Your task is to write a **local MCP server** (in Python) that exposes a few tools Claude can call to **search and fetch recipes** from the public **TheMealDB** API. You'll run and debug it with the MCP Inspector, then wire it into **Claude Desktop** so you can ask for recipes in plain English.

This assignment uses **no API keys you need to create** TheMealDB allows a test key of 1 for education/dev, and it publishes simple JSON endpoints. (themealdb.com)

What you'll build

A local MCP server named meals that provides these tools:

1. **search_meals_by_name(query, limit=5)**
 - Call TheMealDB **Search meal by name** endpoint: `.../search.php?s=<query>`
 - Return up to limit meals with: name, area (cuisine), category, thumbnail, and the meal **id** for drill-down. (themealdb.com)
2. **meals_by_ingredient(ingredient, limit=12)**
 - Call **Filter by main ingredient** : `.../filter.php?i=<ingredient>`
 - Return small cards (name, thumbnail, id). (themealdb.com)
3. **random_meal()**
 - Call **Lookup a single random meal** : `.../random.php`. (themealdb.com)

The API base is `https://www.themealdb.com/api/json/v1/1/...` and the published “test key” is 1, which is sufficient for this homework. (themealdb.com)

Set up your environment

1. Install the MCP Python SDK with CLI :

- With **pip** : pip install "mcp[cli]"
([GitHub](#))

2. You'll write a single Python file (e.g., meals_server.py) that defines your tools and runs FastMCP over **STDIO** .

Important logging rule: For STDIO servers, **never write to stdout** (that corrupts the JSON-RPC stream). Log to **stderr** using a logger. ([Model Context Protocol](#)) **I/O:**

Input → Output shapes you should implement:

1. search_meals_by_name(query, limit)

- **Input:** query: str, limit: int (1–25)
- **Output (list):** objects with { id, name, area, category, thumb }

2. meals_by_ingredient(ingredient, limit)

- **Input:** ingredient: str, limit: int
- **Output (list):** objects with { id, name, thumb }

3. meal_details(id)

- **Input:** id: str | int
- **Output (object):** { id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}] }

4. random_meal()

- **Input:** none
- **Output:** same shape as meal_details.

● Error handling:

- If an endpoint returns "meals": null (no results), return a **clear, empty result** with a short message (e.g., “no matches”).

- Network or JSON errors should raise a clean error to the client (MCP Inspector will display it).

Where in the API to look

- Search by **name** : `.../search.php?s=Arrabiata`
- List by **first letter** : `.../search.php?f=a` (optional bonus)
- **Lookup by id** : `.../lookup.php?i=52772`
- **Random** : `.../random.php`
- **Filter by main ingredient** : `.../filter.php?i=chicken_breast`
- **List categories** : `.../categories.php` (optional bonus)
All of the above are documented on the official API page and work with the dev key 1.
(themealdb.com)

Run and debug locally with MCP Inspector

1. In your project folder, open a terminal and start **development mode** with the server file you wrote:

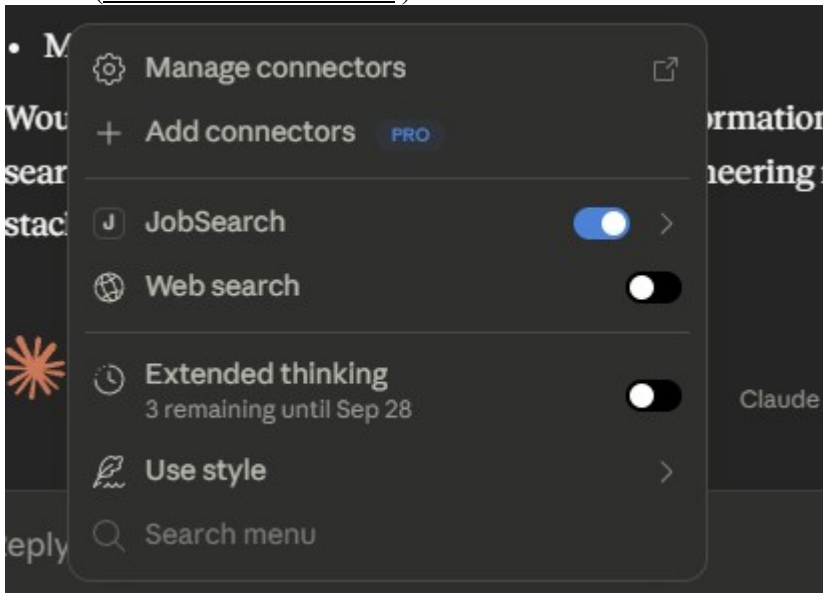
```
None  
mcp dev <your_file_name.py>
```

2. The Inspector opens in your browser. Use it to:
 - Call each tool with test inputs (e.g., "Arrabiata", ingredient "chicken").
 - Verify outputs look like your designed schema.
 - Fix any errors before moving on.
The Inspector is the official debugging UI for MCP servers. ([Model Context Protocol](https://mcp-inspector.com))

Connect your server to Claude Desktop

Claude for Desktop can launch and talk to your local MCP server once you add it to the **Claude config**. The official “Build an MCP server” guide walks through this flow and shows the exact config file location and format. Key points:

- Edit `claude_desktop_config.json` (path shown in the docs for Windows and macOS).
- Add an entry under `mcpServers` with a command (e.g., `python` or `uv`) and the **absolute path** to your project and server file in args.
- Save the file and **restart Claude Desktop** ; look for the tools slider icon to confirm the server is loaded. ([Model Context Protocol](#))



Something like this (Jobsearch in this case)

The doc also notes Windows path tips (use absolute paths; escape backslashes) and includes screenshots of the Claude UI showing tools once the server is configured. ([Model Context Protocol](#))

What to show (prompts to try in Claude)

Once your server is connected and visible in Claude:

- “Find 3 **Italian** pasta recipes by name and show short summaries with images.”
- “List meals that use **chicken** as a main ingredient; include meal IDs so I can ask for details.”

- “Show full details for meal id **52772** .”
- “Give me one **random** meal with ingredients and measures.”

Claude will choose the right tool based on your natural-language question; you can inspect tool calls in the Claude UI. (The workflow of client → tools → results is summarized in the MCP quickstart.) ([Model Context Protocol](#))

1. Screenshot(s) and code snippet :

- Claude Desktop showing your server’s tools and a sample query/answer.

- **TheMealDB API (official docs & endpoints)** — test key 1, sample URLs. (themealdb.com)
- **Build an MCP server (official guide)** — end-to-end tutorial, Claude Desktop config, logging rules, and screenshots. ([Model Context Protocol](#))
- **MCP Python SDK (GitHub README)** — installing mcp[cli], and using **mcp dev** with Inspector. ([GitHub](#))

MCP Inspector (docs) — features and usage for debugging servers. (Model Context Protocol)